Ranveer Thakur Chand, John Nasir, Trevor Oakum, Freddy Rodriguez

Professor Viswanatha Rao

ITAI 1371

11 July 2026

Reflective Journal

Ranveer Thakur Chand:

For this midterm, I found the dataset first on Kaggle, accidently sent our picked dataset to Professor Rao through message instead of submitting it on the assignment, created all the documents and GitHub folder for this project, wrote the imports, loaded and split the data into train/test, applied SMOTE to balance our training set, and finally added the before/after visualization. I first found the Heatwave Dataset (Rajasthan, India, 2006–2025) on Kaggle after going through a few options using Gemini to help narrow down my search. It had a clear binary target and gave us a real problem to solve once Professor Rao told us how imbalanced it was. The imports cell was easy after all the labs we have done. Pandas, numpy, matplotlib, seaborn, kagglehub, and the sklearn/imblearn are the tools we needed for the rest of the notebook. Coded that myself, didn't use AI for this part. Loading the dataset and splitting it 70/30 right away, before any cleaning or exploring, was probably the most important part of the project and of my part in this notebook in my opinion. The instructions were clear about the test set never being touched by preprocessing, and splitting first was the only way to go about it. Used Gemini for this cell with the prompt: "Write code that runs .info(), .describe(), and isnull().sum() on a training DataFrame, then prints value_counts and percentages for a target column called HEATWAVE, and plots the class distribution using seaborn's countplot." Added an AI disclaimer above the cell aswell.

The other main piece was SMOTE. Professor Rao gave us feedback that the raw data was about 95% "no heatwave" and 5% "heatwave," which would let a model just guess "no" every time and still look accurate. SMOTE fixes that by generating synthetic examples of the minority class, applied only to the training set, never the test set. Added a before/after plot so the fix is visible. Used Gemini here too, disclosed with the prompt I used. The visualization code scaling and normalization histograms. I built myself without AI, to see the transformations working instead of just trusting the numbers like I learned in this class and my CV class aswell. Building the pipeline of this project made the order of operations, helped me understand more than just knowing the concepts. One mistake I caught was MinMaxScaler running on top of already scaled data instead of the raw values. The plot looked off; I traced it back with the help of Gemini, and fixed it by keeping a clean copy of the pre-transform data before scaling was applied to it. Then I was responsible for compiling everybody's reflections, downloading all the documents, uploading them to GitHub, and turning it into Canvas.

John Nasir:

During this mid term project, my main responsibility was completing the handling and some technical parts of the data preprocessing. I focused on steps like missing values and feature engineering. These were the parts that needed careful attention to avoid data leakage and to not modify the dataset manually. Working through the steps helped me understand how much the order of preprocessing matters, especially when splitting the dataset before doing anything else.

One thing I learned is that small simple parts of code can break a model if they’re done in the wrong order. So, I had to make sure the scale was fit only on the training data, and that SMOTE was applied only to the training set. This detail is the difference between a correct workflow and one that leaks information.

Another challenge was writing clear Markdown explanations. I used AI to sound more organized, but main code was done by us. I made sure the explanations matched what I actually did in the notebook and followed the professor's guidelines. Adding the AI disclaimer also helped make everything transparent.

Overall, this project gave me a better understanding of how raw data becomes usable for machine learning. By the end, I felt more confident in preparing datasets, checking for issues, and making sure everything is processed correctly before training a model. These skills will help a lot when we move into the finals and start building actual predictive models with this dataset.

Trevor Oakum:

The data we used for this assignment gave me insight on how this data could be misinterpreted by simple mistakes in the data. In the sections that I did (Class Distribution before balancing, One-hot encoding and scaling numeric features), learning how to separate data from one another correctly without corrupting it tells you. Now, in the case with any large dataset, one thing I do always notice is the fear that the data itself was initially collected incorrectly, thus, having data that may not reflect the actual data out there. However, not seeing how this is a problem, considering the data has been verified, we can confidentially train a model without having to worry about feeding it incorrect data. At the sections I worked on, I had an important question answered about something mentioned. As per the assignment's instructions, we were not allowed to change the raw data itself; this begged the question: what if we assigned the raw text (in this case, district), numerical number (1-9 going off of the number of districts), and translate them after it's finished. After seeing how the Encoding worked, I can now understand why. Not only is it "simpler" to have the raw data be in its original format, but it also allows the model to examine the data separately without worrying about corruption by introduced factors.

I did enjoy this midterm, probably more than the labs because we were allowed to collaborate on a higher level and get creative in our approach to trying to interpret the data, and see how we could make improvements in the future, using our logic, and giving us a more sense of reward of a finished product, due to the personal achievement of us building the model from scratch.

Freddy Rodriguez:

This midterm I owned three parts: Getting the data in, doing the first pass to see what we were actually working with, and making sure everything looked ok at the end before we submitted it.

The first thing I did was pull in the Heatwave dataset from Kaggle using kagglehub. This is a simple step, but I checked the shape and column names to make sure it loaded correctly — if that was wrong then all the modelling done on top would have been wrong too.

Next was the initial inspection. Ran. info(),. To confirm that the data was free of missing values, we performed a.describe() and checked on how the classes were distributed within the HEATWAVE class column. That is where I saw 95% no-heatwave and 5% heatwave, which was in practical terms what Professor Rao was explaining to me. This decision with SMOTE — while I didn't quite understand it before, seeing it on the plot made perfect sense in a way that simply hearing about it had not.

Before submitting the machine learning modeling task, I completed the final round of compliance checks: I confirmed that the training set had undergone balanced standardization, while the test set remained in its original, unaltered state. After verifying the dimensions of both datasets and spot-checking their samples, I saved the files in CSV format, and all settings met the specified requirements. This project reminded me not to just ask AI to code for me, run the code and assume it worked. Instead taking my time to compare shapes and counts between steps is what actually catches mistakes early.